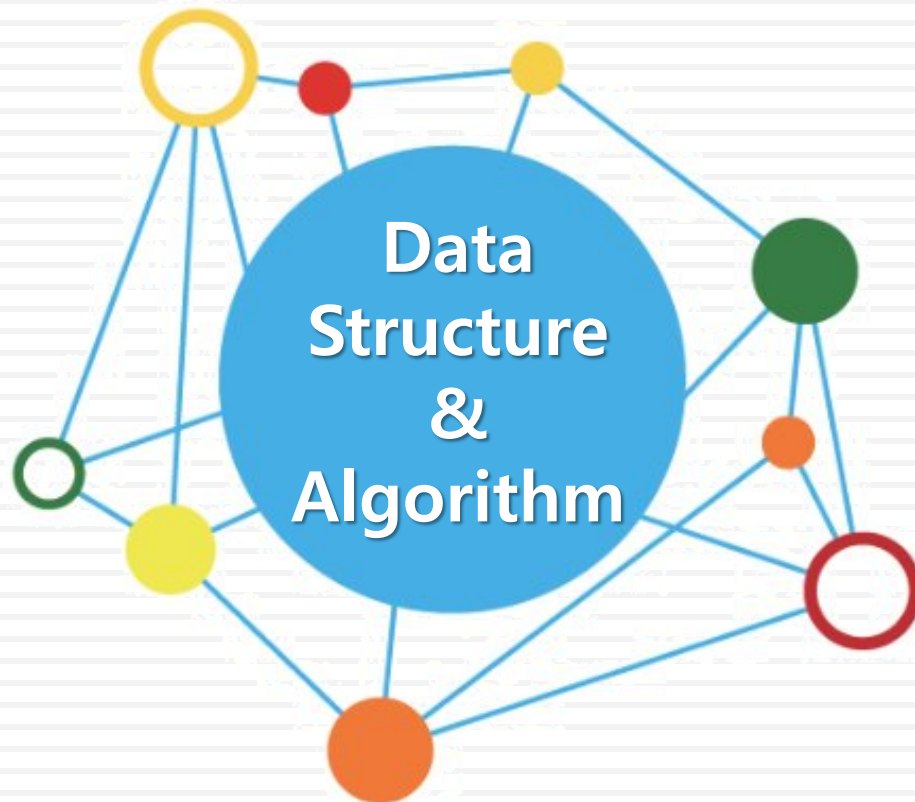


# 데이터 구조론



유길현

연결 데이터구조와  
연결 리스트

## ❖ 순차 데이터구조의 문제점

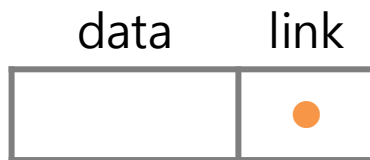
- 삽입 연산이나 삭제 연산 후에 연속적인 물리 주소를 유지하기 위해서 원소들을 이동시키는 추가 작업과 시간 소요
  - 원소들의 이동 작업으로 인한 오버헤드로 원소의 개수가 많고 삽입 · 삭제 연산이 많이 발생하는 경우에 성능상의 문제 발생
- 순차 데이터구조는 배열을 이용해 구현하기 때문에 배열이 갖고 있는 메모리 사용의 비효율성 문제를 그대로 가짐
- 순차 데이터구조에서의 연산 시간에 대한 문제와 저장 공간에 대한 문제를 개선한 데이터 구조 표현 방법 필요

## ❖ 연결 데이터구조(Linked Data Structure)

- 자료의 논리적인 순서와 물리적 순서가 불일치
  - 각 원소에 저장되어 있는 다음 원소의 주소에 의해 순서가 연결되는 방식
    - ☞ **물리적인 순서를 맞추기 위한 오버헤드가 발생하지 않음**
  - 여러 개의 작은 공간을 연결하여 하나의 전체 자료구조를 표현
    - ☞ **크기 변경이 유연하고 더 효율적으로 메모리를 사용**
- 연결 리스트
  - 리스트의 연결 자료구조로 표현
  - 연결하는 방식에 따라 **단순 연결 리스트**와 **원형 연결 리스트**, **이중 연결리스트**, **이중 원형 연결 리스트**

## ❖ 연결 리스트의 노드

- 연결 자료구조에서 하나의 원소를 표현하기 위한 단위 구조
- <원소, 주소>의 구조



노드의 논리적 구조

| 데이터 필드(data field)                                                                                 | 링크 필드(link field)                                                                               |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• 원소의 값을 저장</li><li>• 저장할 원소의 형태에 따라 하나 이상의 필드로 구성</li></ul> | <ul style="list-style-type: none"><li>• 다음 노드의 주소를 저장</li><li>• 포인터 변수를 사용하여 주소 값을 저장</li></ul> |

## ❖ 연결 리스트의 이해

- 선형 리스트 week의 순차 리스트 표현
  - 리스트 week=(월, 화, 수, 목, 금, 토, 일)



(a) 논리적 구조

선형 리스트 week의 순차 리스트 표현



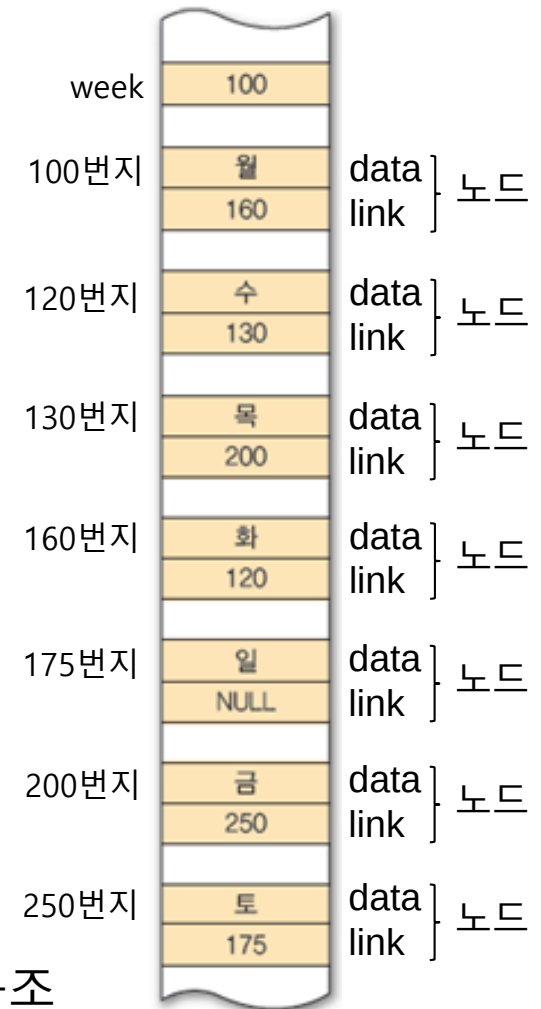
(b) 물리적 구조

## ❖ 연결 리스트의 이해

### ■ 선형 리스트 week의 연결 리스트 표현



(a) 논리적 구조

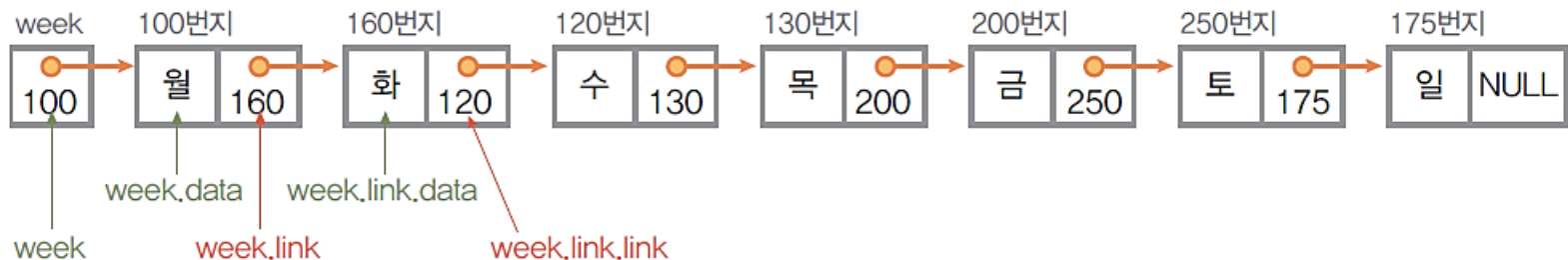


(b) 물리적 구조

## ❖ 연결 리스트의 이해

### ■ 선형 리스트와 연결 리스트 비교

- 리스트 이름 week : 연결 리스트의 시작을 가리키는 포인터 변수
  - ☞ week는 연결 리스트의 첫 번째 노드를 가리킴과 동시에 연결된 리스트 전체 의미
- 연결 리스트의 마지막 노드의 링크 필드 : 노드 끝을 표시하기 위해 NULL(널) 저장
- 공백 연결 리스트 : 포인터 변수 week에 NULL 저장(널 포인터)
- 각 노드의 필드에 저장한 값은 점 연산자를 사용해 액세스



선형 리스트 week의 연결 리스트 구조

## ❖ 연결 리스트의 이해

- 리스트 week의 노드에 대한 구조체 정의  
<원소, 링크>

```
typedef struct Node{  
    char data[4];  
    struct Node*link;  
};
```



## ❖ 단순 연결 리스트(singly linked list)

- 노드가 하나의 링크 필드에 의해서 다음 노드와 연결되는 구조
- 연결 리스트, 선형 연결 리스트(linear linked list)
- 단순 연결 선형 리스트(singly linked linear list)

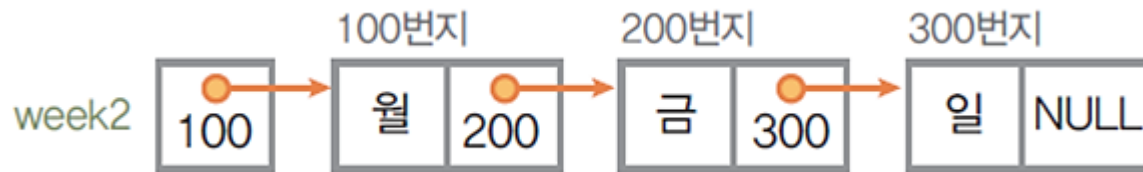
## ❖ 단순 연결 리스트에 노드를 삽입하는 방법

- ① 삽입할 노드를 준비한다.
- ② 새 노드의 데이터 필드에 값을 저장한다.
- ③ 새 노드의 링크 값을 저장한다.
- ④ 리스트의 앞 노드에 새 노드를 연결한다.

## ❖ 단순 연결 리스트 노드 삽입 방법

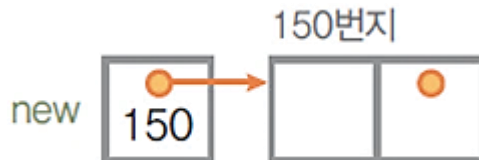
- 단순 연결 리스트 week2=(월, 금, 일)에서 '월' 과 '금' 사이에 '수' 삽입과정

✓ 초기 상태



단순 연결 리스트 week2에서 '수' 노드를 삽입하기 전의 초기 상태

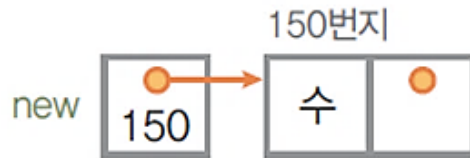
- ① 삽입할 노드 준비 : 공백 노드를 가져와 포인터 변수 new가 가리키게 함



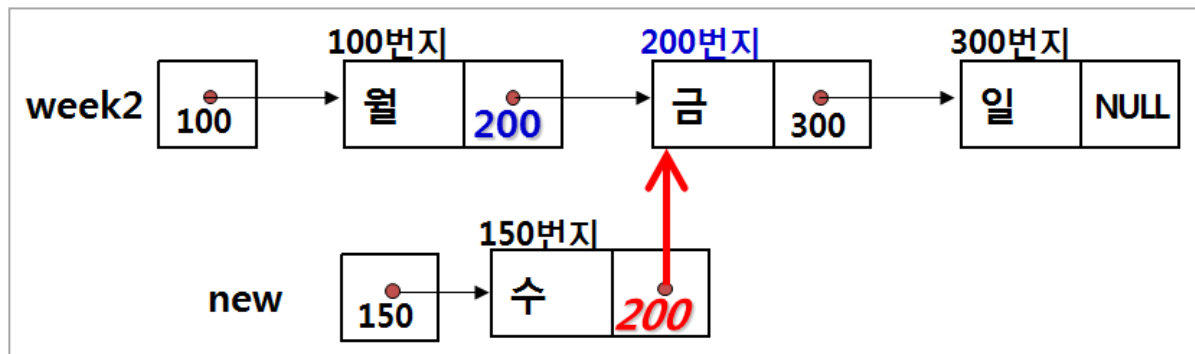
## ❖ 단순 연결 리스트 노드 삽입 방법

- 단순 연결 리스트 week2=(월, 금, 일)에서 '월' 과 '금' 사이에 '수' 삽입과정

② 새 노드의 데이터 필드 값 저장 : new 데이터 필드에 '수'를 저장



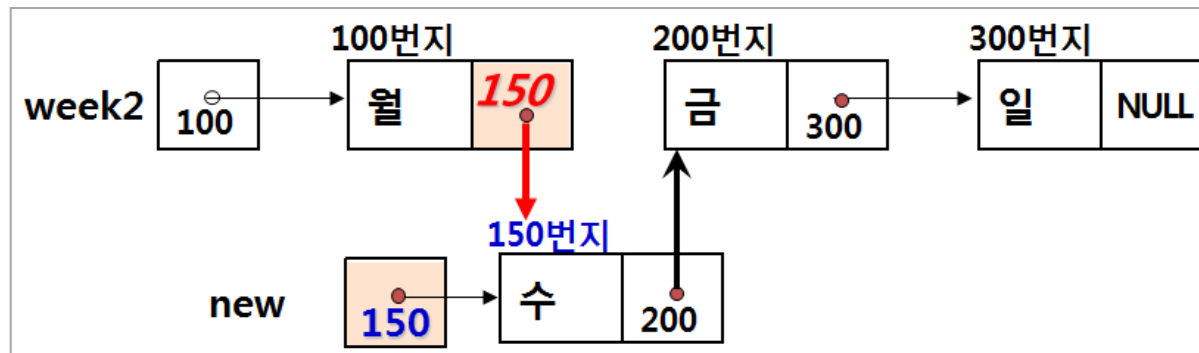
③ 새 노드의 링크 필드 값 지정 : new의 앞 노드 즉 '월'노드의 링크 필드 값을 new 링크 필드에 저장



## ❖ 단순 연결 리스트 노드 삽입 방법

- 단순 연결 리스트 week2=(월, 금, 일)에서 '월' 과 '금' 사이에 '수' 삽입과정

④ 리스트의 앞 노드에 새 노드 연결 : new의 값을 '월' 노드의 링크 필드에 저장



## ❖ 첫 번째 노드 삽입 알고리즘

### 알고리즘

### 단순 연결 리스트의 첫 번째 노드 삽입

insertFirstNode(L, x)

① new  $\leftarrow$  getNode( );

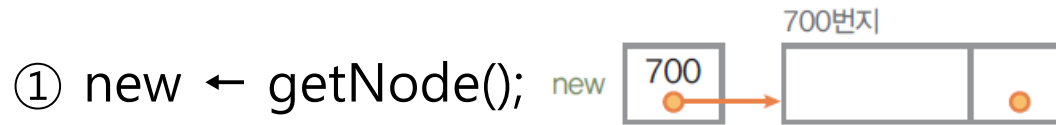
② new.data  $\leftarrow$  x;

③ new.link  $\leftarrow$  L;

④ L  $\leftarrow$  new;

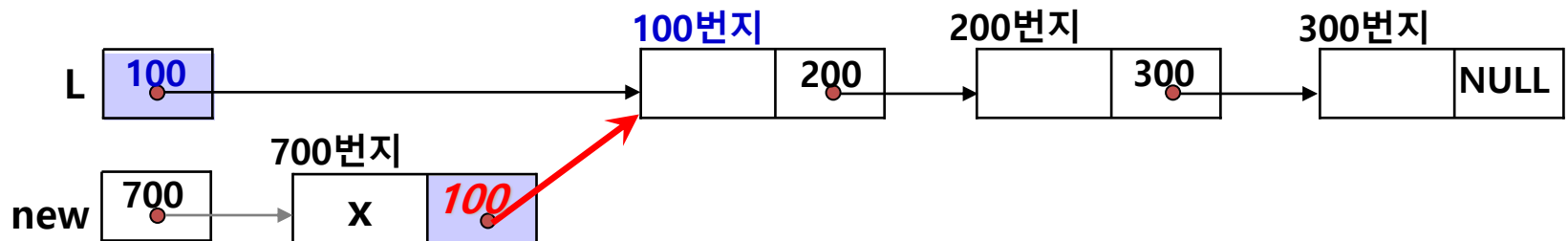
end insertFirstNode()

## ❖ 첫 번째 노드 삽입 알고리즘



③  $new.link \leftarrow L;$

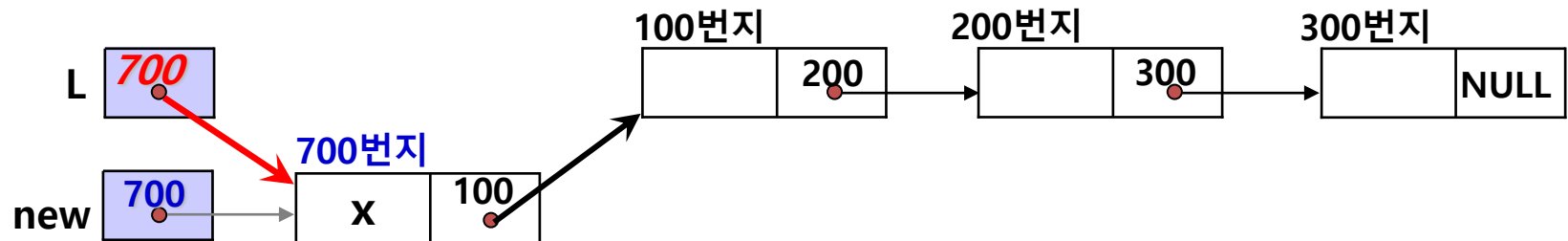
삽입할 노드를 연결하기 위해서 **리스트의 첫 번째 노드 주소(L)**를 삽입할 새 노드 **new의 링크 필드(new.link)**에 저장하여, 새 노드 new가 리스트의 첫 번째 노드를 가리키게 한다.



## ❖ 첫 번째 노드 삽입 알고리즘

④  $L \leftarrow \text{new};$

리스트의 첫 번째 노드 주소를 저장하고 있는 **포인터 L**에, **새 노드의 주소 new**를 저장하여, 포인터 L이 새 노드를 첫 번째 노드로 가리키도록 지정



## ❖ 첫 번째 노드 삽입연산 코딩

```
// 첫 번째 노드로 삽입하는 연산
void insertFirstNode(linkedList_h* L, char *x)
{
    listNode* newNode;
    // 삽입할 새 노드 할당
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy_s(newNode->data, x); // 새 노드의 데이터 필드에 x 저장
    newNode->link = L->head;
    L->head = newNode;
}
```



## ❖ 중간 노드 삽입 알고리즘

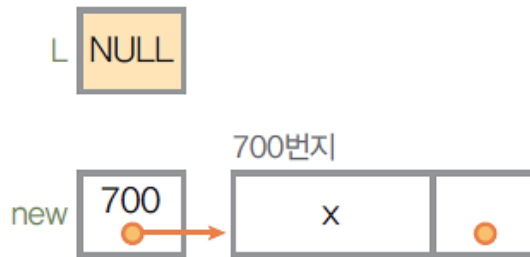
### 알고리즘

### 단순 연결 리스트의 중간 노드 삽입

```
insertMiddleNode(L, pre, x)
  ① new ← getNode( );
  ② new.data ← x;
  ③ {
    if (L = NULL) then {
      ③-① L ← new;
      ③-② new.link ← NULL;
    }
    else{
      ④ {
        ④-① new.link ← pre.link;
        ④-② pre.link ← new;
      }
    }
  }
end insertMiddleNode()
```

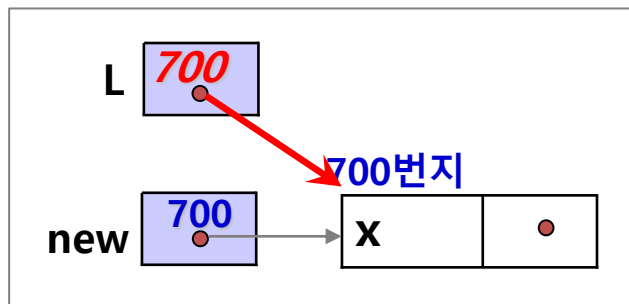
## ❖ 중간 노드 삽입 알고리즘

- ①  $new \leftarrow getNode();$
- ②  $new.data \leftarrow x;$
- ③ 공백리스트인 경우



- ③ - ①  $L \leftarrow new;$

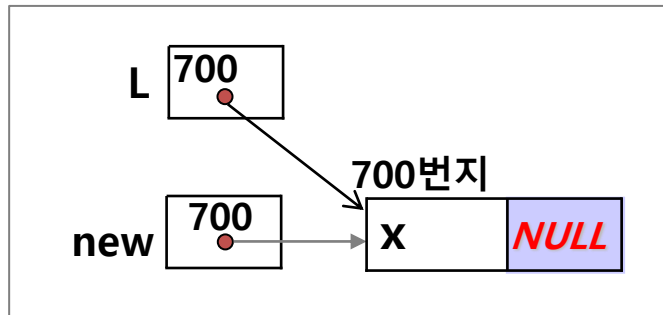
리스트 포인터 **L**에 새 노드 **new**의 주소를 저장하여, 새 노드 **new**가 리스트의 첫 번째 노드가 되도록 함



## ❖ 중간 노드 삽입 알고리즘

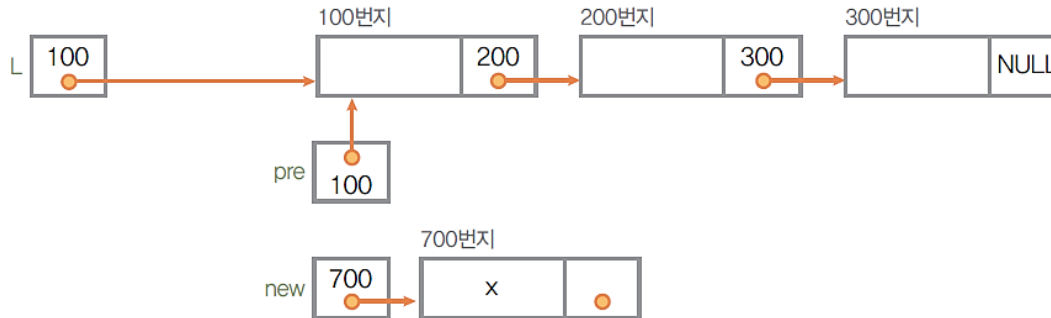
③ - ⑥ new.link ← NULL;

리스트의 마지막 노드인 **new**의 링크 필드에 NULL을 저장해  
마지막 노드임을 표시



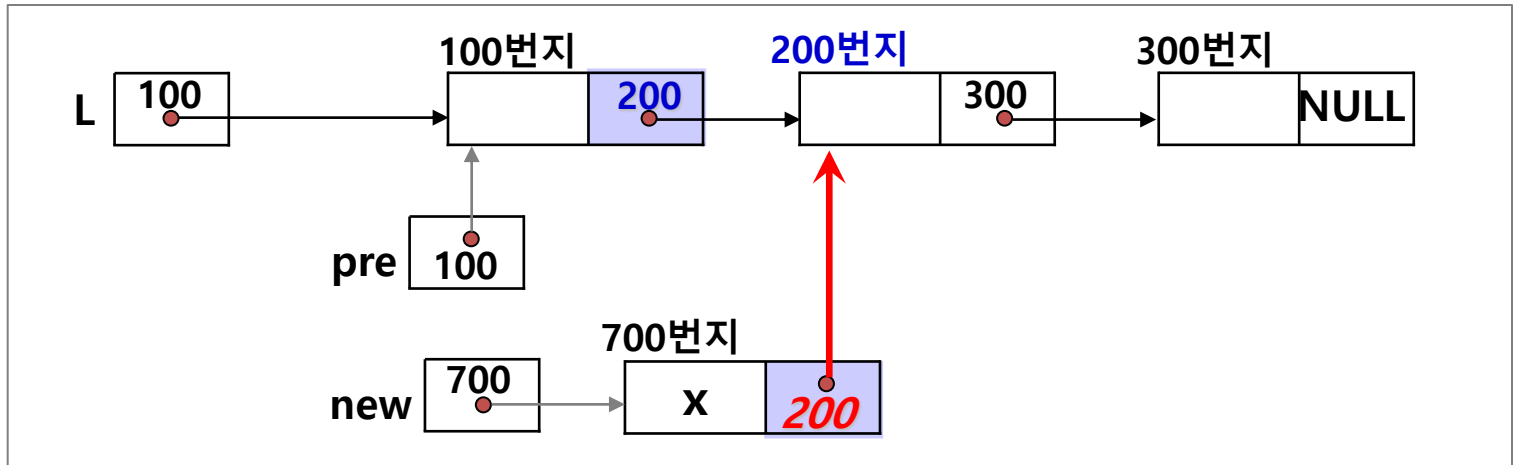
## ❖ 중간 노드 삽입 알고리즘

### ④ 공백리스트가 아닌 경우



### ④ - ㉠ new.link

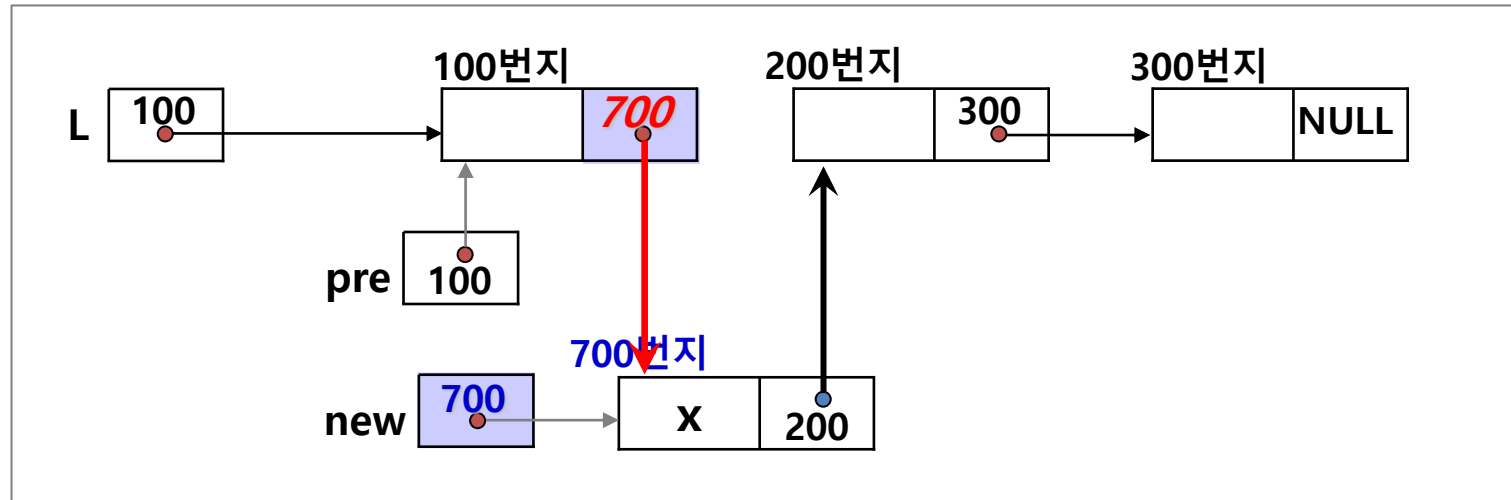
노드 pre의 링크 필드 값을 노드 new의 링크 필드에 저장하여,  
새 노드 new가 노드 pre의 다음 노드를 가리키도록 함



## ❖ 중간 노드 삽입 알고리즘

④ - ⑥ pre.link ← new;

포인터 new의 값을 노드 pre의 링크 필드에 저장하여, 노드 pre가 새 노드 new를 다음 노드로 가리키도록 함



## ❖ 중간 노드 삽입연산 코딩

```
// 중간 노드로 삽입하는 연산
void insertMiddleNode(linkedList_h* L, listNode *pre, char*x)
{
    listNode* newNode;
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, x);
    if (L == NULL) // 공백 리스트인 경우
    {
        newNode->link = NULL; // 새 노드를 첫 번째이자 마지막 노드로 연결
        L->head = newNode;
    }
    else if (pre == NULL) // 삽입 위치를 지정하는 포인터 pre가 NULL인 경우
    {
        L->head = newNode; // 새 노드를 첫 번째 노드로 삽입
    }
    else
    {
        newNode->link = pre->link; // 포인터 pre의 노드 뒤에 새 노드 연결
        pre->link = newNode;
    }
}
```

## ❖ 마지막 노드 삽입 알고리즘

### 알고리즘

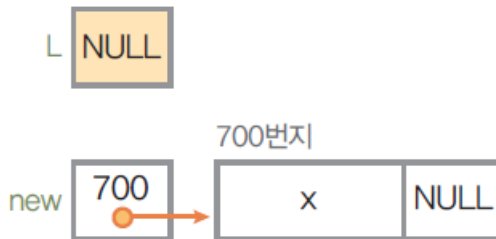
### 단순 연결 리스트의 마지막 노드 삽입

insertLastNode(L, x)

```
    ① new ← getNode( );
    ② new.data ← x;
    ③ new.link ← then;
    ④ {
        if(L==NULL) then{
            ④-① L ← new;
            return;
        }
    }
    ⑤ {
        ⑤-① temp ← L;
        while(temp.link == NULL) do
            ⑤-② temp ← temp.link;
        ⑤-③ temp.link ← new;
    }
end insertLastNode()
```

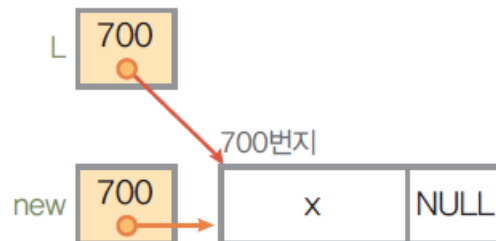
## ❖ 마지막 노드 삽입 알고리즘

- ①  $new \leftarrow getNode();$
- ②  $new.data \leftarrow x;$
- ③  $new.link \leftarrow NULL;$
- ④ 공백 리스트인 경우



- ④-①  $L \leftarrow new;$

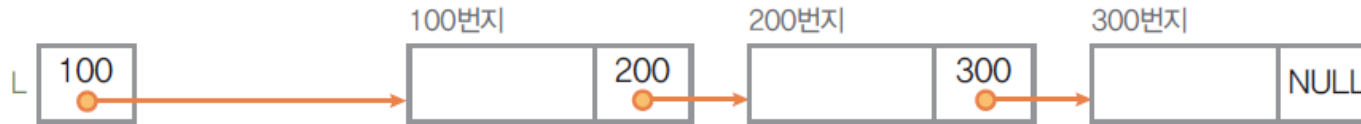
리스트 포인터  $L$ 에 새 노드  $new$ 의 주소( $700$ ) 저장.  $new$ 는 리스트  $L$ 의 첫 번째 노드이자 마지막 노드





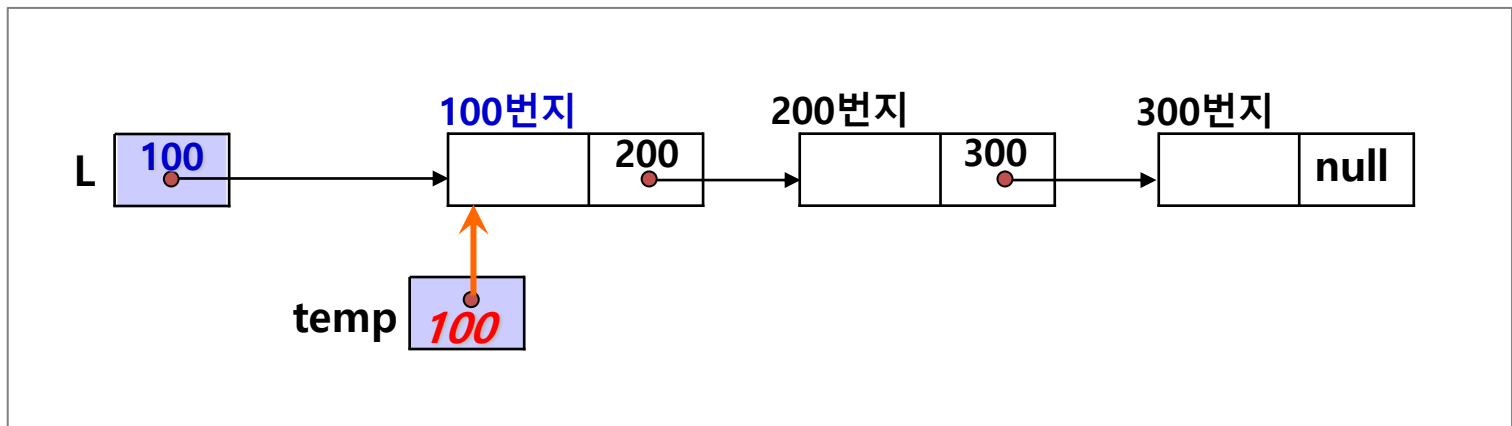
## ❖ 마지막 노드 삽입 알고리즘

### ⑤ 공백 리스트가 아닌 경우



⑤-① temp ← L;

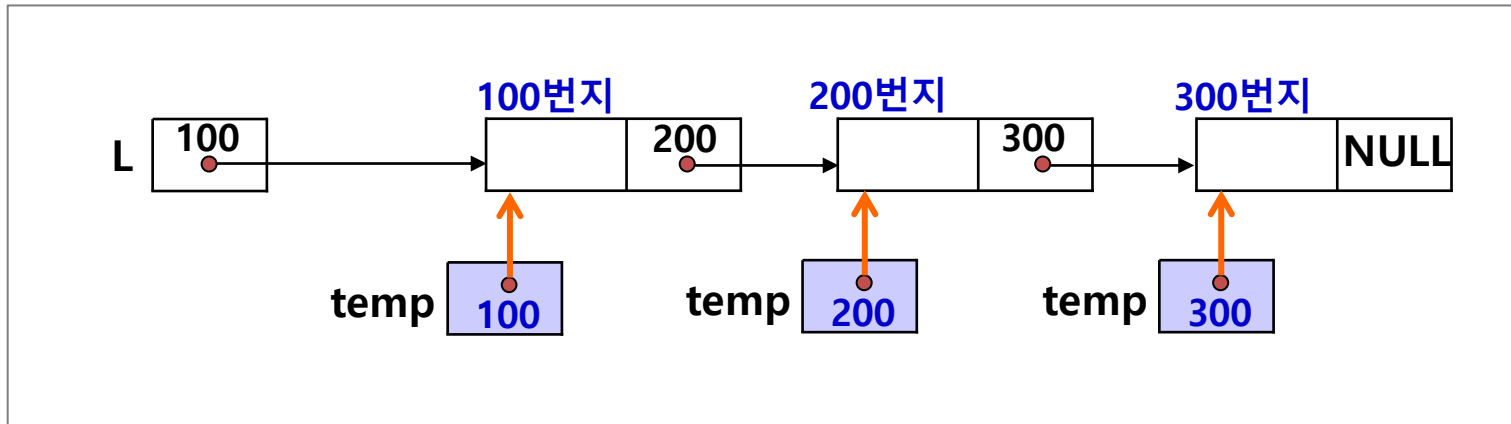
현재 리스트 L의 마지막 노드를 찾기 위해서 노드를 순회할 임시 포인터 temp에 리스트의 첫 번째 노드의 주소(L)를 지정



## ❖ 마지막 노드 삽입 알고리즘

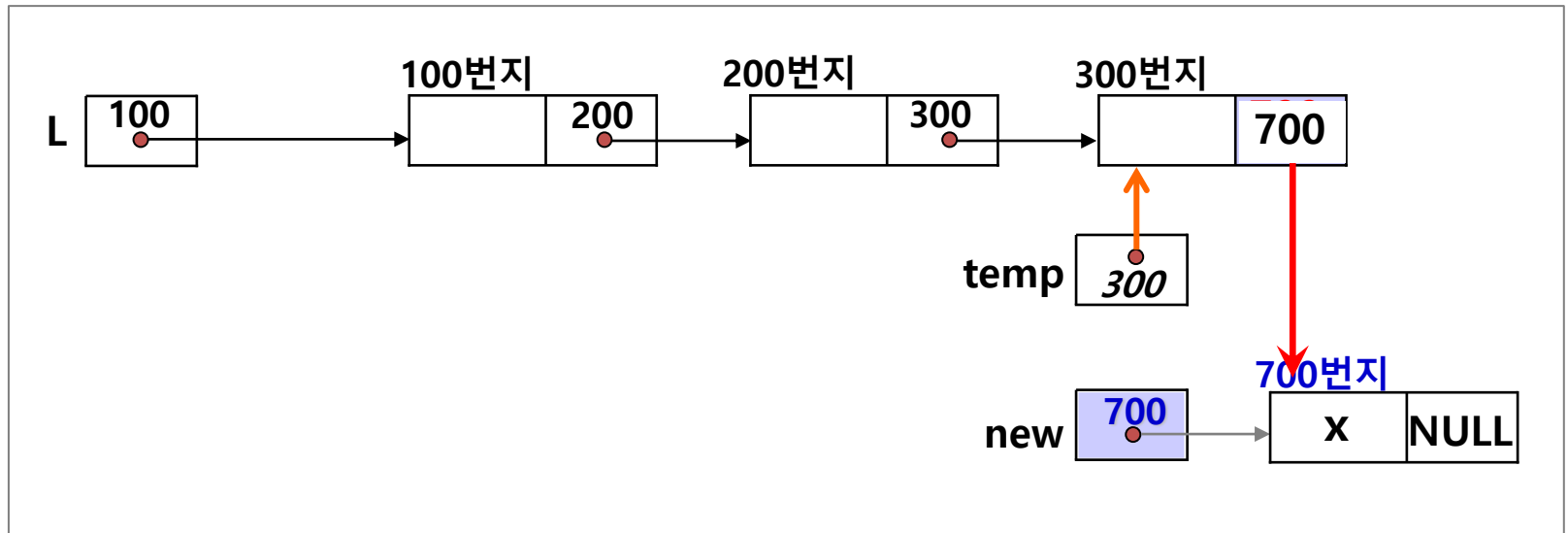
⑤-⑥ temp ← temp.link;

순회 포인터 temp가 가리키는 노드의 링크 필드가 NULL이 아닌 동안(while (temp.link ≠ NULL)) 링크 필드를 따라 이동. 링크 필드가 NULL인 노드 즉, 마지막 노드를 찾으면 while 문을 끝내고 ⑤-③를 수행



## ❖ 마지막 노드 삽입 알고리즘

⑤-③ temp.link ← new;



## ❖ 마지막 노드 삽입연산 코딩

```
// 마지막 노드로 삽입하는 연산
void insertLastNode(linkedList_h* L, char* x)
{
    listNode* newNode;
    listNode* temp;
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy_s(newNode->data, x);
    newNode->link = NULL;
    if (L->head == NULL)           // 현재 리스트가 공백인 경우
    {
        L->head = newNode;        // 새 노드를 리스트의 시작 노드로 연결
        return;
    }
    // 현재 리스트가 공백이 아닌 경우
    temp = L->head;
    while (temp->link != NULL)
        temp = temp->link; // 현재 리스트의 마지막 노드를 찾음
    temp->link = newNode; // 새 노드를 마지막 노드(temp)의 다음 노드로 연결
}
```

## ❖ 연결 리스트 노드 구조를 구조체로 선언

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 단순 연결 리스트의 노드 구조를 구조체로 정의
typedef struct ListNode {
    char data[4];
    struct ListNode* link;
} ListNode;

// 리스트 시작을 나타내는 head 노드를 구조체로 정의
typedef struct {
    ListNode* head;
} linkedList_h;
```

## ❖ 공백 리스트 생성

```
// 공백 연결 리스트를 생성하는 연산
linkedList_h* createLinkedList_h(void)
{
    linkedList_h* L;
    L = (linkedList_h*)malloc(sizeof(linkedList_h));
    L->head = NULL;
    return L;
}
```

## ❖ 첫 번째 노드 삽입

```
// 첫 번째 노드로 삽입하는 연산
void insertFirstNode(linkedList_h* L, char *x)
{
    listNode* newNode;
    // 삽입할 새 노드 할당
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, x); // 새 노드의 데이터 필드에 x 저장
    newNode->link = L->head;
    L->head = newNode;
}
```

## ❖ 마지막 노드 삽입

```
// 마지막 노드로 삽입하는 연산
void insertLastNode(linkedList_h* L, char* x)
{
    listNode* newNode;
    listNode* temp;
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, x);
    newNode->link = NULL;
    if (L->head == NULL)           // 현재 리스트가 공백인 경우
    {
        L->head = newNode;        // 새 노드를 리스트의 시작 노드로 연결
        return;
    }
    // 현재 리스트가 공백이 아닌 경우
    temp = L->head;
    while (temp->link != NULL)
        temp = temp->link; // 현재 리스트의 마지막 노드를 찾음
    temp->link = newNode; // 새 노드를 마지막 노드(temp)의 다음 노드로 연결
}
```



## ❖ 연결 리스트의 전체 메모리를 해제하는 연산

```
// 연결 리스트의 전체 메모리를 해제하는 연산
void freeLinkedList_h(linkedList_h* L)
{
    listNode* p;
    while (L->head != NULL)
    {
        p = L->head;
        L->head = L->head->link;
        free(p);
        p = NULL;
    }
}
```

## ❖ 연결 리스트를 순서대로 출력

```
void printList(linkedList_h* L)
{
    listNode* p;
    printf("L=");
    p = L->head;
    while (p != NULL)
    {
        printf("%s", p->data);
        p = p->link;
        if (p != NULL)
            printf(", ");
    }
    printf(") \n");
}
```

## ❖ 프로그램 동작을 위한 메인 함수 만들기

```
int main()
{
    linkedList_h* L;
    L = createLinkedList_h();
    printf("(1) 공백 리스트 생성하기! \n");
    printList(L);
    getchar();

    printf("(2) 리스트에 [수] 노드 삽입하기! \n");
    insertFirstNode(L, "수");
    printList(L);
    getchar();

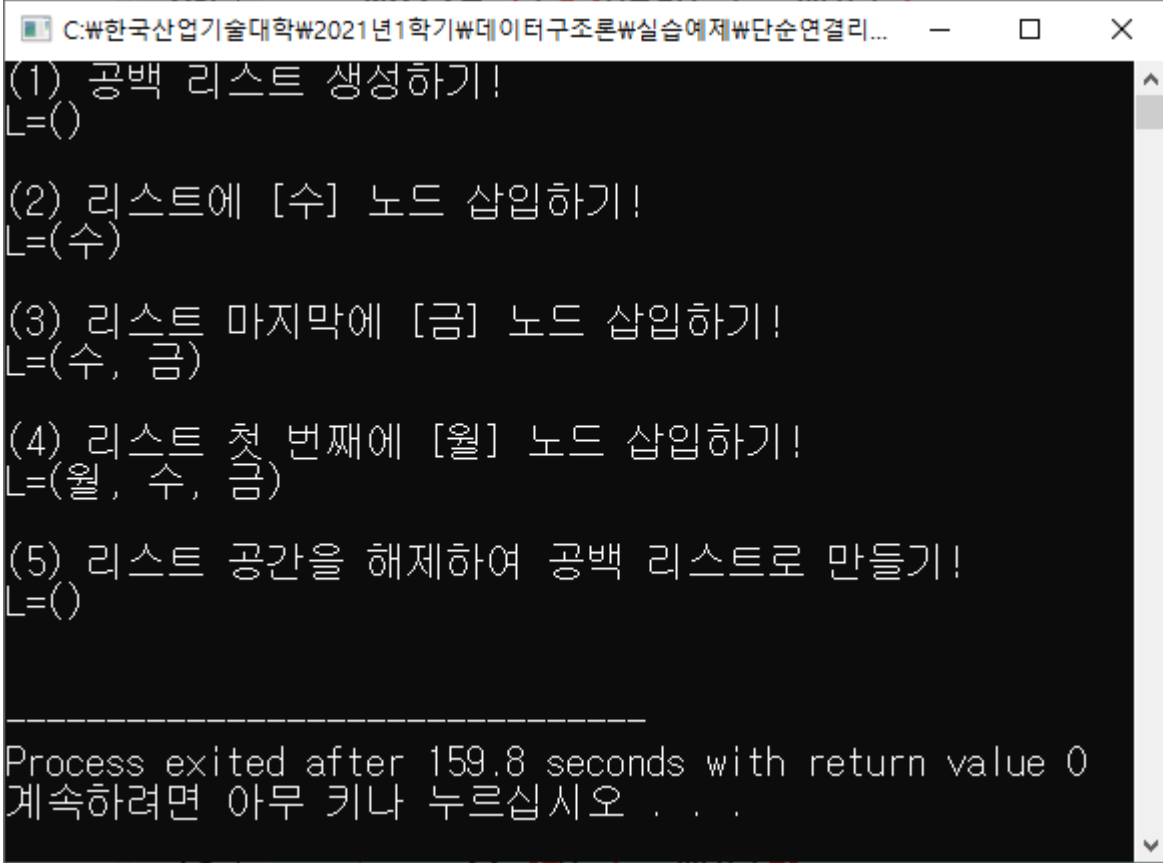
    printf("(3) 리스트 마지막에 [금] 노드 삽입하기! \n");
    insertLastNode(L, "금");
    printList(L);
    getchar();
}
```

## ❖ 프로그램 동작을 위한 메인 함수 만들기

```
printf("(4) 리스트 첫 번째에 [월] 노드 삽입하기! \n");  
insertFirstNode(L, "월");  
printList(L);  
getchar();  
  
printf("(5) 리스트 공간을 해제하여 공백 리스트로 만들기! \n");  
freeLinkedList_h(L);  
printList(L);  
  
getchar();  
  
return 0;  
}
```

## ❖ 단순 연결 리스트 프로그램 동작 확인 : [교재 169p](#)

### ■ 실행 결과



```
C:\₩한국산업기술대학₩2021년1학기₩데이터구조론₩실습예제₩₩단순연결리...
(1) 공백 리스트 생성하기!
L=()

(2) 리스트에 [수] 노드 삽입하기!
L=(수)

(3) 리스트 마지막에 [금] 노드 삽입하기!
L=(수, 금)

(4) 리스트 첫 번째에 [월] 노드 삽입하기!
L=(월, 수, 금)

(5) 리스트 공간을 해제하여 공백 리스트로 만들기!
L=()

-----
Process exited after 159.8 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .
```

# 단순 연결 리스트 노드 삭제

## ❖ 단순 연결 리스트 노드 삭제 방법

- ① 삭제할 노드의 앞 노드를 찾는다.
- ② 앞 노드에서 삭제할 노드의 링크 필드값을 저장한다.
- ③ 삭제한 노드의 앞 노드와 삭제한 노드의 다음 노드를 연결한다.

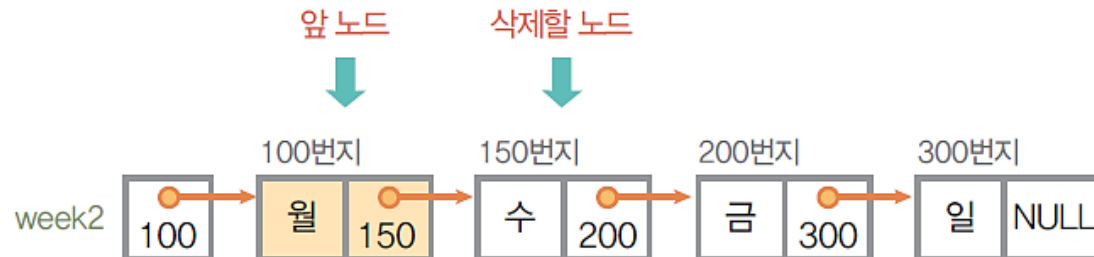
- 단순 연결 리스트 week2=(월, 수, 금, 일)에서 원소 '수' 삭제 과정

✓ 초기 상태



단순 연결 리스트 week2에서 '수' 노드를 삭제하기 전의 초기 상태

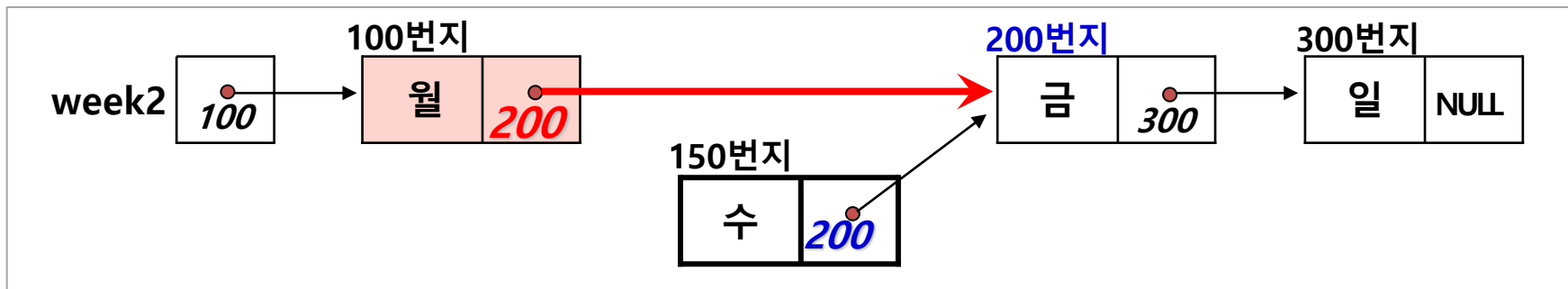
- ① 앞 노드를 찾음 : 삭제할 원소의 앞 노드(선행자)를 찾음



# 단순 연결 리스트 노드 삭제

## ❖ 단순 연결 리스트 노드 삭제 방법

- 단순 연결 리스트 week2=(월, 수, 금, 일)에서 원소 '수' 삭제 과정
  - ② 앞 노드에 삭제할 노드의 링크 필드값 저장 : 삭제할 원소 '수'의 링크 필드 값을 앞 노드의 링크 필드에 저장
  - ③ 삭제한 노드의 앞뒤 노드 연결 : 삭제한 노드의 앞 노드인 '월' 노드를 삭제한 노드의 다음 노드인 '금' 노드에 연결



## ❖ 노드 삭제 알고리즘

- 리스트 L에서 포인터 pre가 가리키는 노드의 다음 노드 삭제 알고리즘 : 포인터 old(삭제할 노드)

### 알고리즘

### 단순 연결 리스트의 노드 삭제

deleteNode(L, pre)

```
① if (L == NULL) then error;  
   else {  
       ② {  
           ②-a old ← pre.link;  
           ②-b if (old == NULL) then return;  
           ②-c pre.link ← old.link;  
           ②-d returnNode(old);  
       }  
   }
```

end deleteNode()



## ❖ 노드 삭제 알고리즘

### ■ 삭제 연산 수행 과정

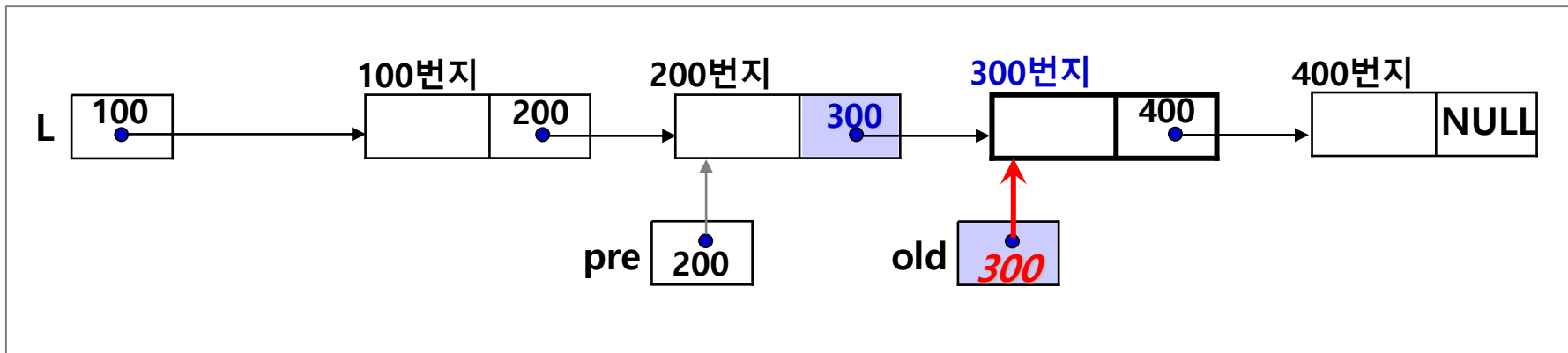
#### ① 공백 리스트인 경우

공백 연결 리스트 L이 공백이면 오류 처리한다.

#### ② 공백 리스트가 아닌 경우

#### ②-① old ← pre.link;

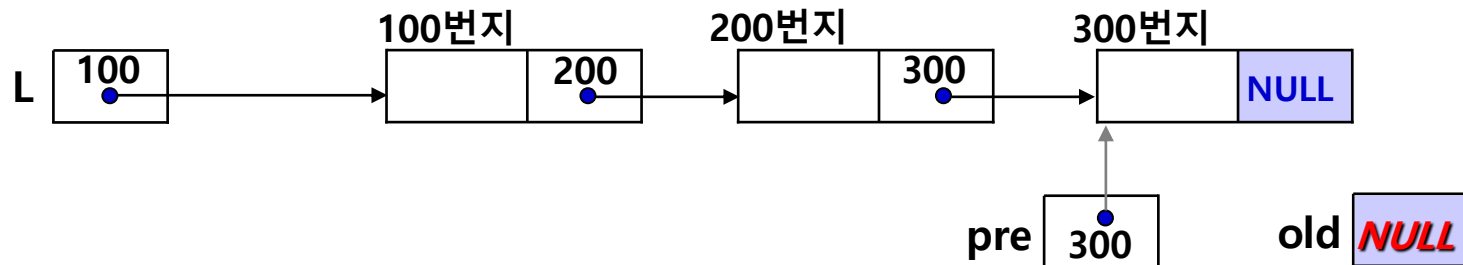
노드 pre의 다음노드의 주소(pre.link)를 포인터 old에 저장하여, 포인터 old가 다음 노드를 가리키게 한다.



## ❖ 노드 삭제 알고리즘

- 삭제 연산 수행 과정
  - ② 공백 리스트가 아닌 경우
  - ②-⑥ if (old == NULL) then return;  
만약 노드 pre가 리스트의 마지막 노드였다면

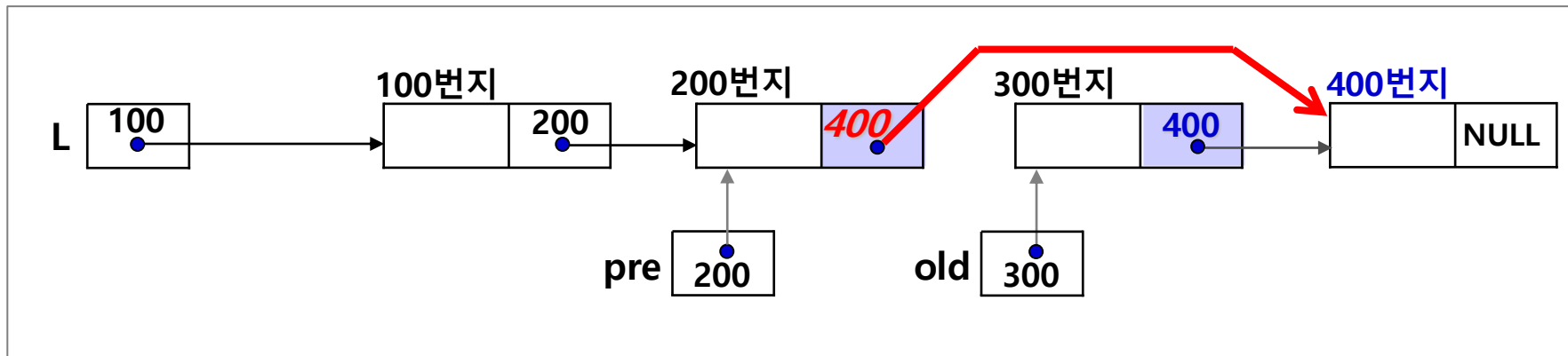
포인터 pre가 가리키는 노드가 마지막 노드인 경우에,  
②-⑥  $old \leftarrow pre.link;$  를 수행한 상태 :



⇒ pre.link의 값은 null이므로 포인터 old의 값은 NULL이 된다.  
결국 노드 pre 다음에 삭제할 노드가 없다는 의미이므로 삭제 연산을 수행하지 못하고 반환(return).

## ❖ 노드 삭제 알고리즘

- 삭제 연산 수행 과정
  - ② 공백 리스트가 아닌 경우
  - ②-③  $\text{pre.link} \leftarrow \text{old.link};$



- ②-④  $\text{returnNode}(\text{old});$   
삭제한 노드 old의 메모리를 반환

## ❖ 노드 삭제연산 코딩(1/2)

```
void deleteNode(linkedList_h* L, listNode* p)
{
    // 삭제할 노드의 선행자 노드를 나타낼 포인터
    listNode* pre;

    // 공백 리스트라면 삭제 연산 중단
    if (L->head == NULL)
        return;

    // 리스트에 노드가 한 개만 있는 경우
    if (L->head->link == NULL)
    {
        free(L->head);           // 첫 번째 노드를 메모리에서 해제하고
        L->head = NULL;          // 리스트 시작 포인터를 NULL로 설정
        return;
    }
}
```

## ❖ 노드 삭제연산 코딩(2/2)

```
// 삭제할 노드가 없다면 삭제 연산 중단
else if (p == NULL)
    return;
// 삭제할 노드 p의 선행자 노드를 포인터 pre를 이용해 찾음
else
{
    pre = L->head;
    while (pre->link != p)
    {
        pre = pre->link;
    }
    // 삭제할 노드 p의 선행자 노드와 다음 노드를 연결
    pre->link = p->link;
    // 삭제 노드의 메모리 해제
    free(p);
}
```

## ❖ 노드 탐색 알고리즘

### 알고리즘

### 단순 연결 리스트의 노드 탐색

searchNode(L, x)

① temp  $\leftarrow$  L;

② { while (temp  $\neq$  NULL) do {  
    ②-① if (temp.data == x) then return temp;  
    ②-② else temp  $\leftarrow$  temp.link;  
}

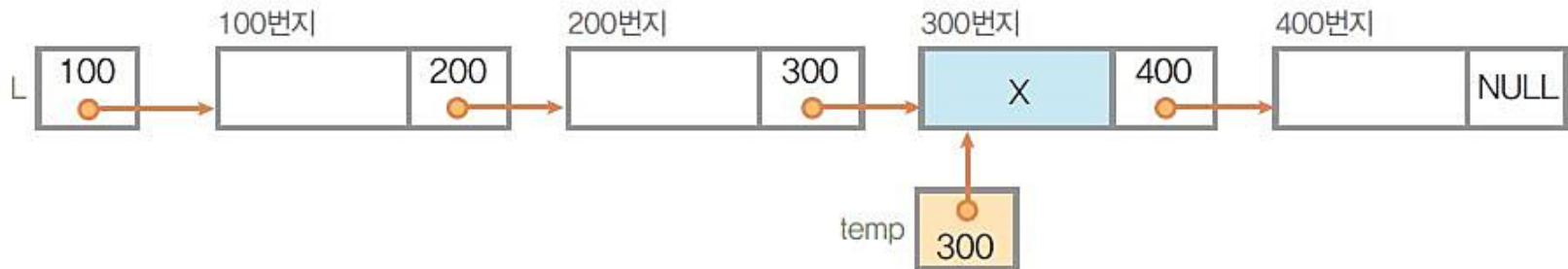
③ return temp;

end searchNode()

## ❖ 노드 탐색 알고리즘

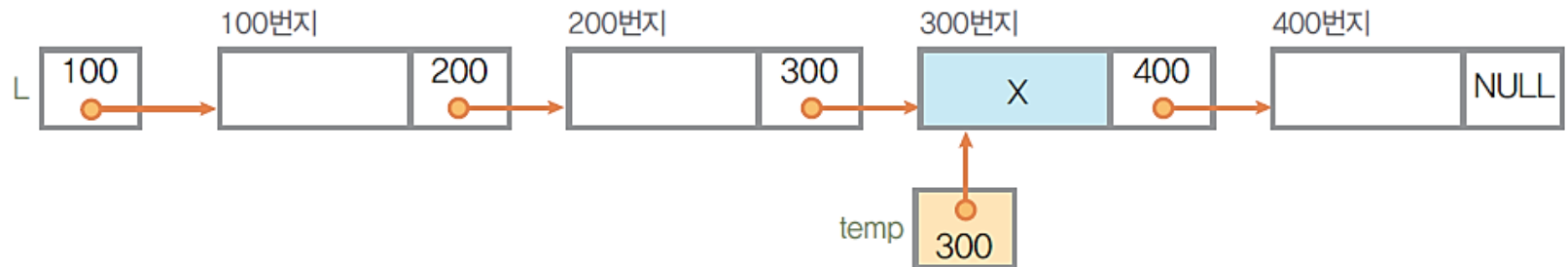
- 단순 연결 리스트에서 x노드 탐색 과정

①  $temp \leftarrow L$ ;



② 순회 포인터가 NULL이 아닌 경우

②-① if(temp.data == x) then return temp;

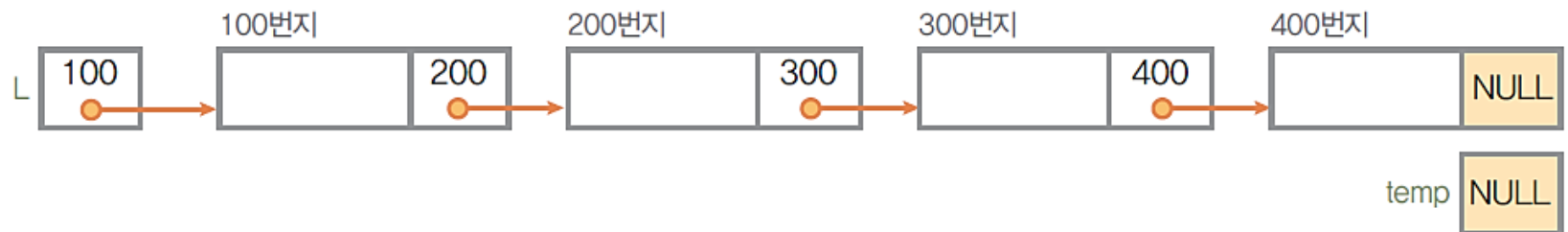


## ❖ 노드 탐색 알고리즘

- 단순 연결 리스트에서 x노드 탐색 과정

②-⑥ else temp ← temp.link;

③ return temp;





## ❖ 노드 탐색 연산 코딩

```
// 리스트에서 x 노드를 탐색하는 연산
listNode* searchNode(linkedList_h* L, char* x)
{
    listNode *temp;
    temp = L->head;
    while (temp != NULL)
    {
        if (strcmp(temp->data, x) == 0)
            return temp;
        else
            temp = temp->link;
    }
    return temp;
}
```

## ❖ 리스트의 노드 순서를 역순으로 바꾸는 코딩(1/2)

```
// 리스트의 노드 순서를 역순으로 바꾸는 연산
void reverse(linkedList_h * L)
{
    listNode* p;
    listNode* q;
    listNode* r;

    p = L->head;      // 포인터 p를 첫 번째 노드에 설정
    q = NULL;
    r = NULL;
```

## ❖ 리스트의 노드 순서를 역순으로 바꾸는 코드(2/2)

```
// 리스트의 첫 번째 노드부터 링크를 따라 다음 노드로 이동하면서  
// 노드 간의 연결을 바꿈  
while (p != NULL)  
{  
    r = q;  
    q = p;  
    p = p->link;  
    q->link = r;  
}  
L->head = q;  
}
```

## ❖ 단순 연결리스트 노드 삽입, 탐색 삭제, 역순으로 바꾸기 구현(1/3)

```
int main()
{
    linkedList_h* L;
    listNode *p;
    L = createLinkedList_h();          // 공백 리스트 생성
    printf("(1) 리스트에 [월], [수], [일] 노드 삽입하기! \n");
    insertLastNode(L, "월");
    insertLastNode(L, "수");
    insertLastNode(L, "일");
    printList(L);
    getchar();
}
```

## ❖ 단순 연결리스트 노드 삽입, 탐색 삭제, 역순으로 바꾸기 구현(2/3)

```
printf("(2) 리스트에서 [수] 노드 탐색하기! \n");
p = searchNode(L, "수");
if (p == NULL)
    printf("찾는 데이터가 없습니다.\n");
else
    printf("[%s]를 찾았습니다.\n", p->data);
getchar();

printf("(3) 리스트의 [수] 뒤에 [금] 노드 삽입하기! \n");
insertMiddleNode(L, p, "금");
printList(L);
getchar();
```

### ❖ 단순 연결리스트 노드 삽입, 탐색 삭제, 역순으로 바꾸기 구현(3/3)

```
printf("(4) 리스트에서 [일] 노드 삭제하기! \n");
p = searchNode(L, "일"); // 삭제할 노드 위치 p를 찾음
deleteNode(L, p);        // 포인터 p 노드 삭제
printList(L);
getchar();

printf("(5) 리스트 순서를 역순으로 바꾸기! \n");
reverse(L);
printList(L);

freeLinkedList_h(L);      // 리스트의 메모리 해제
getchar();

return 0;
}
```

## ❖ 단순 연결 리스트 프로그램2 동작 확인 : [교재 174p](#)

### ■ 실행 결과

```
C:\한국산업기술대학교\2021년1학기\데이터구조론\실습예제\단순연결...
(1) 리스트에 [월], [수], [일] 노드 삽입하기!
L=(월, 수, 일)

(2) 리스트에서 [수] 노드 탐색하기!
[수]를 찾았습니다.

(3) 리스트의 [수] 뒤에 [금] 노드 삽입하기!
L=(월, 수, 금, 일)

(4) 리스트에서 [일] 노드 삭제하기!
L=(월, 수, 금)

(5) 리스트 순서를 역순으로 바꾸기!
L=(금, 수, 월)

-----
Process exited after 18.11 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .
```

## ❖ 단순연결 리스트를 이용하여 영화 제목과 개봉 년도를 입력하고 출력하는 프로그램을 만들어 보자

- 초기 화면 출력은 아래와 같이 한다.

```
-----  
1. 영화 정보 추가  
2. 영화 정보 출력  
3. 종료  
-----  
번호를 선택하세요 : █
```

- 1번을 선택하여 영화 제목과 개봉 년도를 입력한다. 새로운 입력을 위해서는 1번을 다시 선택한다.
- 2번 항목을 누르면 영화 제목과 제작 년도가 같은 줄에 출력 되게 한다.
- 3번 항목을 누르면 동적으로 생성된 메모리 공간을 삭제한다.
- 프로그램에 필요한 함수는 앞에 제시된 함수를 사용한다.



## ❖ 연결 리스트의 노드 구조를 만든다

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#define SIZE 40

// 단순 연결 리스트의 노드 구조를 구조체로 정의
typedef struct ListNode {
    char data[SIZE];
    int year;
    struct ListNode* link;
} ListNode;

// 리스트 시작을 나타내는 head 노드를 구조체로 정의
typedef struct {
    ListNode* head;
} linkedList_h;
```

## ❖ 공백 연결 리스트를 생성한다.

```
// 공백 연결 리스트를 생성하는 연산
linkedList_h* createLinkedList_h(void)
{
    linkedList_h* L;
    L = (linkedList_h*)malloc(sizeof(linkedList_h));
    L->head = NULL;
    return L;
}
```

## ❖ 연결 리스트의 전체 메모리를 해제한다.

```
// 연결 리스트의 전체 메모리를 해제하는 연산
void freeLinkedList_h(linkedList_h* L)
{
    listNode* p;
    while (L->head != NULL)
    {
        p = L->head;
        L->head = L->head->link;
        free(p);
        p = NULL;
    }
}
```

## ❖ 연결 리스트를 순서대로 출력한다.

```
void printList(linkedList_h* L)
{
    int Num = 0;
    listNode* p;          // = NULL;
    p = L->head;
    while (p != NULL)
    {
        Num++;
        printf(" %d 제목: %s\\t", Num, p->data);
        printf(" 년도: %d\\n", p->year);
        p = p->link;
    }
}
```

## ❖ 연결 리스트의 삽입 연산 기능을 구현한다.

```
void insertNode(linkedList_h* L, char *s, int y)
{
    listNode* newNode;
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, s);
    newNode->year = y;

    // 공백 리스트인 경우
    // 새 노드를 첫 번째이자 마지막 노드로 연결
    if (L->head == NULL)
    {
        newNode->link = NULL;
        L->head = newNode;
    }
}
```

## ❖ 연결 리스트의 삽입 연산 기능을 구현한다.

```
// 선행 노드가 있으면 그 다음 노드에 삽입되게 한다.  
else if (newNode == NULL)  
{  
    newNode->link=L->head; // 새 노드를 첫 번째 노드로 삽입  
    L->head = newNode;  
}  
else //리스트 중간에 삽입되게 한다.  
{  
    newNode->link = L->head->link;  
    L->head->link = newNode;  
}  
}
```

## ❖ 메뉴 선택을 위한 정보를 화면에 출력한다.

```
int menu()
{
    int men = 0;
    printf(" -----\\n");
    printf("  1. 영화 정보 추가\\n");
    printf("  2. 영화 정보 출력\\n");
    printf("  3. 종료\\n");
    printf(" -----\\n");
    printf(" 번호를 선택하세요 : ");
    scanf("%d", &men);
    return men;
}
```

## ❖ 프로그램 실행을 위해 메인 함수를 만든다.

```
int main()
{
    int cho = 0, y;
    char s[40];
    linkedList_h* L;
    L = createLinkedList_h();

    while ((cho = menu()) != 3)
    {
        getchar();
        if (cho == 1)
        {
            printf(" 영화의 제목을 입력하세요 : ");
            scanf("%s", &s);
            printf(" 영화의 개봉 연도를 입력하세요 : ");
            scanf("%d", &y);
            insertNode(L, s, y);
        }
    }
}
```



## ❖ 프로그램 실행을 위해 메인 함수를 만든다.

```
    else if (cho == 2)
    {
        printList(L);
        getchar();
    }
    else if (cho == 3)
    {
        freeLinkedList_h(L);
        break;
    }
    system("cls");
}
return 0;
}
```